Reporting COBOL

REPORTE DE APRENDIZAJE TÉCNICO COBOL

13 marzo 2026 |

1.1. Introducción

Durante el periodo de aprendizaje reciente se ha trabajado en el fortalecimiento de conocimientos relacionados con el lenguaje COBOL y su ejecución en entornos Linux basados en Debian, enfocándose principalmente en el procesamiento de archivos, generación de reportes estructurados y organización modular del código mediante rutinas y subprogramas.

El objetivo principal ha sido comprender cómo funcionan los sistemas tradicionales de procesamiento de datos empresariales, los cuales siguen siendo ampliamente utilizados en sectores como banca, seguros y sistemas corporativos.

A lo largo de este proceso se realizaron ejercicios prácticos que involucraron:

- Lectura y escritura de archivos secuenciales
- Generación de reportes con formato tipo tabla
- Manipulación de cadenas y alineación de datos
- Uso de estructuras de control del lenguaje
- Implementación de subprogramas mediante llamadas CALL
- Consulta de información en archivos externos
- Compilación y ejecución de programas COBOL en Linux

© 2026 NTT DATA, Inc.

1.2. Estructura del Lenguaje COBOL

Se revisó la arquitectura base de los programas COBOL, los cuales se organizan en divisiones bien definidas que separan la configuración del entorno, la definición de datos y la lógica del programa.

1.2.1 Estructura de un programa COBOL

Se revisó la estructura básica que compone un programa COBOL.

- IDENTIFICATION DIVISION: Define el nombre del programa y su identificación.
- ENVIRONMENT DIVISION: Configura el entorno de ejecución y los archivos.
- DATA DIVISION: Contiene la definición de variables, estructuras y archivos.
- PROCEDURE DIVISION: Implementa la lógica del programa.

Esta estructura permite mantener una separación clara entre configuración, datos y lógica, lo cual facilita el mantenimiento en sistemas empresariales grandes.

1.2.2 Definición y Manejo de Variables

Se trabajó con diferentes tipos de variables utilizando la cláusula PIC, la cual define el formato y longitud de los datos.

- PIC X
- PIC 9
- PIC A

```
01 WS-CLAVE PIC 9(08).
01 WS-NOMBRE PIC X(40).
01 WS-GRADO PIC 9(02).
01 WS-CARRERA PIC 9(02).
```

También se utilizaron variables auxiliares para operaciones de formato como:

- variables de longitud.
- posiciones dentro de cadenas.
- buffers temporales para construir líneas de reporte.

1.3. Manejo de Archivos en COBOL

Uno de los temas principales estudiados fue el manejo de archivos secuenciales, que constituye una de las funciones más importantes de COBOL.

1.3.1 Declaración de archivos

Los archivos se definen dentro de la FILE SECTION.
Ejemplo:

```
FD ESTUDIANTES-ARCHIVO.
01 ESTUDIANTES-REGISTRO.
        05 EST-REG-CLAVE PIC 9(08).
```

© 2026 NTT DATA, Inc.

```
05 EST-REG-NOMBRE PIC X(40).
05 EST-REG-GRADO PIC 9(02).
```

Esto permite leer o escribir registros estructurados directamente desde archivos de texto.

1.3.2 Modos de apertura de archivos

Durante las prácticas se analizaron los distintos modos de acceso:

- INPUT: Lectura de archivos existentes.
- OUTPUT: Creación o sobreescritura de archivos.
- EXTEND: Agregar registros al final.
- I-O: Lectura y escritura.

1.3.3 Lectura de archivos

La lectura se realiza mediante la instrucción READ, normalmente dentro de ciclos controlados.

Ejemplo conceptual:

```
READ ARCHIVO
        AT END
                SET FIN-ARCHIVO TO TRUE
END-READ
```

Esto permite procesar registros hasta encontrar el final del archivo.

1.4. Generación de Reportes COBOL

Se realizaron ejercicios enfocados en formatear información en estructuras tipo tabla, algo común en sistemas COBOL orientados a reportes.

Se utilizaron dos enfoques principales:

- Construcción dinámica con STRING
- Uso de buffers y posicionamiento de texto

Ejercicios > 03-CALL-AVANZADO > REP-EST-MAT.txt

```
1  ************************************************************************************************************************
2
3                                                REPORTE DE ESTUDIANTES                                  FECHA: 12/03/2026
4
5  ------------------------------------------------------------------------------------------------------------------------
6  PCB999                                                BECA COBOL 2026                                        PAGINA NO.0001
7  ________________________________________________________________________________________________________________________
8
9   CLAVE  |            NOMBRE ALUMNOS              |  GRADO  |  CARRERA  |     MATERIA                    |     PROFESOR
10 ________________________________________________________________________________________________________________________
11 21002814|DIEGO ALEJANDRO MARTINEZ LOPEZ           |   11    |    25     |HISTORIA                        |GABRIELA ALEJANDRA D
12 21002875|SOFIA FERNANDA RAMIREZ TORRES            |   11    |    25     |FISICA                          |MIGUEL ANGEL HERRERA
13 21002812|CARLOS EDUARDO HERNANDEZ GARCIA          |   11    |    25     |INGLES                          |FERNANDO JAVIER ORTE
14 21002859|LUIS ANGEL RODRIGUEZ NAVARRO             |   11    |    25     |INFORMATICA                     |RICARDO ANTONIO GOME
15
16 No. Estudiantes: 04
17 ************************************************************************************************************************
18
```

© 2026 NTT DATA, Inc.

1.4.1 Construcción dinámica con STRING

La instrucción STRING permite concatenar múltiples campos en una sola línea de salida.

Ejemplo conceptual:
STRING
    WS-CLAVE
    "|"
    WS-NOMBRE
    "|"
    WS-GRADO
INTO LINEA-TEMP

```
195         STRING
196             " CLAVE  " DELIMITED BY SIZE
197             "|"
198             PHRASE-NOMBRE DELIMITED BY SIZE
199             "|"
200             "  GRADO  "  DELIMITED BY SIZE
201             "|"
202             "  CARRERA  " DELIMITED BY SIZE
203             "|"
204             COL-MATERIA DELIMITED BY SIZE
205             "|"
206             COL-PROFE DELIMITED BY SIZE
207             "|"
208         INTO LINEA-TEMP.
209         WRITE LINEA-TEMP.
210         PERFORM 100402-L-TABLA-MAKER.
211         EXIT.
```

Posteriormente la línea es escrita al archivo con:
WRITE LINEA-TEMP
Este método permite mayor flexibilidad cuando el formato del reporte puede variar.

1.4.2 Uso de buffers y posicionamiento de texto

Se trabajó con buffers de tamaño fijo (por ejemplo 132 caracteres) para generar reportes con formato alineado.

Ejemplo conceptual:
01 LINEA-TEMP PIC X(132).

Posteriormente se insertan datos en posiciones específicas para lograr:

- alineación
- centrado de valores
- separación por columnas

© 2026 NTT DATA, Inc.

```
COMPUTE LONG-WORD = FUNCTION LENGTH("NOMBRE ALUMNOS").
COMPUTE POS = (40 / 2) - (LONG-WORD / 2).
MOVE "NOMBRE ALUMNOS" TO PHRASE-NOMBRE(POS: LONG-WORD).

COMPUTE LONG-WORD = FUNCTION LENGTH("MATERIA").
COMPUTE POS = (20 / 2) - (LONG-WORD / 2).
MOVE "MATERIA" TO COL-MATERIA(POS: LONG-WORD).

COMPUTE LONG-WORD = FUNCTION LENGTH("PROFESOR").
COMPUTE POS = (20 / 2) - (LONG-WORD / 2).
MOVE "PROFESOR" TO COL-PROFE(POS: LONG-WORD).
```

También se utilizaron variables auxiliares para calcular posiciones dentro de la cadena.

1.4.3 Uso de FILLER para Formato

Se revisó el uso de FILLER para crear estructuras de salida con formato fijo.

Ejemplo conceptual:
01 REPORTE.
  05 CLAVE PIC 9(08).
  05 FILLER PIC X VALUE "|".
  05 NOMBRE PIC X(40).

El uso de FILLER permite:

- insertar separadores
- mantener columnas alineadas
- estructurar reportes de forma más legible

```
01 ENCABEZADOS.
  1 reference
  05 LINEA-TITLE.
    10 FILLER PIC X(55) VALUE SPACES.
    10 L-TEXT-TITULO PIC A(22) VALUE "REPORTE DE ESTUDIANTES".
    10 FILLER PIC X(38) VALUE SPACES.
    10 L-TEXT-FECHA PIC X(7) VALUE "FECHA: ".
    1 reference
    10 L-DATA-FECHA PIC X(10).
```

Este método es muy utilizado en sistemas legacy donde el formato del archivo es estricto.

1.5. Manejo de Archivos en Control de Flujo del Programa

Se revisaron diferentes estructuras de control para manejar la lógica de ejecución.

1.5.1 PERFORM

Permite ejecutar rutinas o repetir bloques de código.

Ejemplo:
PERFORM PROCESAR-DATOS

También se utilizó en ciclos:
PERFORM UNTIL FIN-ARCHIVO

© 2026 NTT DATA, Inc.

```
100-INICIO.
    PERFORM 100100-OPEN-FILES.
    PERFORM 100200-GET-DATE.
    PERFORM 100300-START-VARIABLES.
    PERFORM 100400-HEADER-MAKER.
    PERFORM 100500-READ-FIELDS UNTIL FIN-ARCHIVO.
    PERFORM 100600-CLOSE-FILES.
    EXIT.
```

1.5.2 Condiciones nivel 88 (Switches)

Permite ejecutar rutinas o repetir bloques de código.

Las condiciones 88 se utilizaron para manejar estados lógicos del programa.

Ejemplo:
01 SWITCHES PIC X(02) VALUE "NO".
  88 SW-CONTINUAR VALUE "NO".
  88 SW-DETENER VALUE "SI".

```
WORKING-STORAGE SECTION.
    01 SWITCHES.
        1 reference
        05 WS-FIN PIC XX VALUE "N".
            3 references
            88 FIN-ARCHIVO VALUE "S".
```

Esto facilita la lectura del código al permitir usar condiciones semánticas en lugar de valores literales.

1.6. Rutinas y Modularización del Código

1.6.1 PERFORM

Para mejorar la organización del código se implementaron rutinas internas mediante PERFORM.

Ejemplo:

PERFORM HEADER-MAKER

PERFORM DATA-IN

Esto permite separar responsabilidades dentro del programa, por ejemplo:

- generación de encabezados
- lectura de datos
- generación de reportes

© 2026 NTT DATA, Inc.

1.6.2 Uso de Subprogramas con CALL

Se trabajó con subprogramas externos, los cuales permiten reutilizar lógica en múltiples programas.

Ejemplo de llamada:

```
CALL "SEARCH-MATERIAS-DATA"
USING WS-CLAVE WS-NOMBRE WS-PROFE
```

El subprograma recibe parámetros mediante:

```
LINKAGE SECTION
```

Esto permite que el programa principal envíe datos al subprograma y reciba resultados.

En los ejercicios realizados, esta técnica se utilizó para consultar información de materias desde un archivo externo.

1.6.3 Consulta de Información en Archivos Externos

Se implementó un subprograma encargado de:

1. Abrir el archivo de materias
2. Leer registros secuencialmente
3. Comparar la clave recibida
4. Retornar el nombre de la materia y el profesor

Esto permitió simular una consulta de datos externa, similar a cómo funcionan muchos sistemas empresariales basados en archivos.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. DATA-MATERIAS-SEARCH.

ENVIRONMENT DIVISION.
    INPUT-OUTPUT SECTION.
        FILE-CONTROL.
            View references
            SELECT MAT-DOC ASSIGN TO "materias.txt"
            ORGANIZATION IS LINE SEQUENTIAL.

DATA DIVISION.
    FILE SECTION.
    View references
    FD MAT-DOC.
        01 MAT-REG.
            2 references
            05 MAT-CLAVE PIC 99.
            2 references
            05 MAT-NOMBRE PIC A(20).
            2 references
            05 MAT-PROFE PIC A(20).
```

```
WORKING-STORAGE SECTION.
    01 SWITCHES.
        1 reference
        05 WS-FIN PIC XX VALUE "N".
            3 references
            88 FIN-ARCHIVO VALUE "S".
LINKAGE SECTION.
    2 references
    01 LK-CLAVE PIC XX.
    2 references
    01 LK-NOMBRE PIC A(20).
    2 references
    01 LK-PROFE PIC A(20).

PROCEDURE DIVISION USING LK-CLAVE LK-NOMBRE LK-PROFE.

PERFORM MAIN-PROCEDURE.
1 reference
MAIN-PROCEDURE.
    OPEN INPUT MAT-DOC.

    INITIALIZE MAT-CLAVE.
    INITIALIZE MAT-NOMBRE.
    INITIALIZE MAT-PROFE.

    MOVE "N" TO WS-FIN.

    PERFORM UNTIL FIN-ARCHIVO
        READ MAT-DOC
            AT END
                SET FIN-ARCHIVO TO TRUE
            NOT AT END
                IF MAT-CLAVE = LK-CLAVE
                    MOVE MAT-NOMBRE TO LK-NOMBRE
                    MOVE MAT-PROFE TO LK-PROFE
                    SET FIN-ARCHIVO TO TRUE
                END-IF
        END-READ
    END-PERFORM.
CLOSE MAT-DOC.
GOBACK.
```

© 2026 NTT DATA, Inc.

1.7. Compilación y Ejecución en Debian

1.7.1 PERFORM

Los programas fueron compilados utilizando GnuCOBOL en un entorno Linux.

- Compilación de programa principal:
  - cobc -x programa.cbl
- Compilación de subprograma:
  - cobc -m subprograma.cbl

El programa principal genera un ejecutable, mientras que el subprograma genera un módulo que puede ser cargado dinámicamente mediante CALL.

- generación de encabezados
- lectura de datos
- generación de reportes

1.8. Conclusión

El proceso de aprendizaje permitió adquirir una comprensión sólida de los fundamentos del lenguaje COBOL y su integración en entornos Linux.

Entre los conocimientos más relevantes adquiridos se encuentran:

- estructura del lenguaje COBOL
- manejo de archivos secuenciales
- generación de reportes con formato
- manipulación de cadenas y buffers
- uso de condiciones lógicas con nivel 88
- modularización mediante rutinas
- comunicación entre programas mediante CALL
- compilación y ejecución en entornos Debian

Estos conocimientos constituyen una base importante para continuar profundizando en el desarrollo de aplicaciones COBOL orientadas al procesamiento de datos empresariales.

© 2026 NTT DATA, Inc.

1.9. Materiales / Recursos Utilizados

- Cobol tutorial for Beginners | Cobol Essential Training
- MAINFRAME || COBOL ARRAYS - INTERNAL TABLE, SUBSCRIPT, INDEX, S0C4, BEST EXPLANATION

© 2026 NTT DATA, Inc.